Generating Ensembles of Search Policies to Solve Baba Is You Levels

Robert Ronge, Erik Albrecht, Dominik Woiwode, Alexander Dockhorn
Faculty of EECS
Leibniz University Hannover
Hannover, Germany
{robert.ronge, erik.albrecht}@stud.uni-hannover.de, {woiwode, dockhorn}@tnt.uni-hannover.de

***Abstract*—Baba Is You is a challenging puzzle game that requires agents to not only navigate spatial environments but also dynamically manipulate the game's rules. In this work, we extend prior evolutionary approaches to solving Baba Is You levels by introducing tree-based search policies evolved through genetic programming. These policies leverage a rich set of base heuristics to guide search more effectively than linear combinations used in earlier work. To improve generalization, we construct a minimal set of reusable search policies through integer linear programming and develop ensemble agents that select appropriate policies for new levels using learned classifiers. Our experiments show that tree-based policies outperform linear ones in terms of level coverage, and that nearest neighbor classifiers generalize well to previously unseen levels.**

***Index Terms*—Baba is You, Evolutionary Algorithm, Ensemble Classifier**

I. Introduction

Baba Is You challenges players—and AI agents—by allowing in-game rule manipulation, leading to a vast search space. Olson et al. [1] demonstrated that evolutionary algorithms can evolve effective search policies for solving levels.[1]

In this work, we extend the previously presented evolutionary optimization approach by increasing the potential complexity of search policies. While the previous study was limited to a linear combination of multiple base heuristics, we extend the optimization approach to use genetic programming to find a more complex search policy represented as a tree structure. With increasing complexity, we hope to find search policies for even the hardest levels of the framework.

We further extend the analysis of the search policy optimization process by constructing a meta-agent that selects a suitable search policy based on a level's initial game state. This is done in a multi-stage optimization process. First, we try to find a specialized search policy for each level in the framework. Next, we test the policies in their capability to solve other levels and use integer linear programming to construct a minimal set of heuristics that can be used to solve most levels. Last but not least, we train a classifier that selects a search policy based on the levels' characteristics. While the linear integer programming approach shines light on the potential optimum given a set of search policies, studying the performance of the constructed meta-agent will show how well found search policies generalize across previously unseen levels.

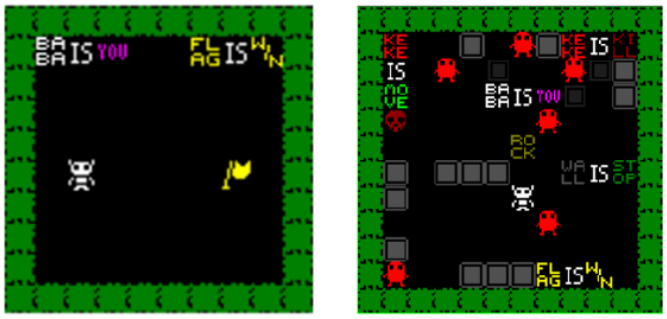

Fig. 1. Two exemplary levels of the Baba is You framework. While a few levels, like the one on the left, just require the agent to interpret the rules, others require active manipulation of the rules to solve the level.

II. Baba is You

Baba Is You is an innovative puzzle video game that challenges traditional puzzle mechanics by allowing players to manipulate the game's rules directly. The game takes place on a grid-based game board, where the player must navigate a character to reach a goal. While it shares similarities with classic Sokoban-like puzzle games, its distinguishing feature is the player's ability to alter the fundamental mechanics of the game world by rearranging rule-defining words within the environment (see Figure 1). This unique mechanic has been widely praised for its originality and depth, earning the game numerous awards and critical acclaim within the gaming community. At the same time, it results in a complex challenge for AI agents due to the enormous game tree complexity this mechanic results in.

In the following, we review the rule modification system, which enables players to dynamically alter how the game operates. Each level consists of a game board populated with tiles containing words. These words represent objects within the game (e.g., "Rock," "Keke," "Flag"), properties (e.g., "Win," "Push," "Defeat"), and the linking word "Is," which can be used to form rules. By repositioning these words, players can change interactions within the game, effectively rewriting the mechanics of each level. Rules are typically structured as three-

Short Paper

[1]We refer to these as search policies rather than heuristics to distinguish them from base heuristics.

word statements, where the middle word must be ”Is” to form a valid rule. These rules can be categorized into prefixes and suffixes: the prefix generally describes an object (e.g., ”Baba Is,” ”Flag Is”), while the suffix assigns a property or action (e.g., ”Is You,” ”Is Win,” ”Is Defeat”). A conventional rule set might include ”Baba Is You” (assigning control of Baba to the player) and ”Flag Is Win” (making the flag the victory condition). The gameplay revolves around manipulating these rules to create or invalidate conditions required to progress through levels.

From an artificial intelligence (AI) perspective, Baba Is You presents significant challenges for automated problem-solving techniques. Traditional Sokoban-style puzzle games already exhibit large search spaces due to the complexity of movement and positioning [2]. However, the additional layer of rule modification in Baba Is You exponentially increases the search space and makes it challenging to solve levels in a limited time frame. With this work, we demonstrate how efficient search policies can be learned to reliably solve most levels of the game.

III. Keke AI Competition and Framework

The Keke AI Competition [3] was launched to advance AI research on Baba Is You. Built on a JavaScript-based framework [4], it provides agents with the level setup and a simulator for forward planning. At the corresponding competition at IEEE CoG'22, Agents were tested under strict time and iteration limits. The top agent achieved a 66% win rate on hidden levels using 10,000 iterations and 10 seconds per level—marking a strong initial benchmark.

To enhance efficiency and compatibility with standard AI libraries, we migrated the framework from JavaScript to Python [5]. This removed the need for a server-client setup, enabled local execution, and allowed multiprocessing for better resource use. We updated the framework to better reflect the final version of the game [6], though without adding new keywords. Originally released at Nordic Game Jam 2017 [7], Baba Is You has since evolved significantly. Due to these changes, some levels and solutions from the original framework are now invalid, slightly reducing baseline agent performance.

IV. Search Policy Optimization

A. Used Base Heuristics

The search policy guides the agent's decisions by ranking successor states for further exploration during the search process. As in previous work by Olson et al. [1], we use an evolutionary algorithm to optimize a search policy that combines several base heuristics. In alignment with the previous study, used base heuristics used are grouped into three categories: (1) object type min-/maximizers, (2) distance-based heuristics, and (3) meta-heuristics. Note that, in contrast to Olson et al., we do not assign different weights at the initialization of each base heuristic. Instead, this will be left to the optimization process. Base heuristics further added or modified by us are marked with an *.

1) Object Type Min-/Maximizers: These heuristics quantify the presence of specific game elements. Including them in the final search policy, the optimizer can reward or penalize states based on their composition:

Number of Goals: Measures the number of goal objects—i.e., objects associated with the `win` property.

Number of Player objects: Counts the number of objects currently assigned the `you` property.

Number of Push objects: Counts the number of objects that can be pushed based on the presence of a `push` rule.

Number of Auto movers: Returns the number of objects with the `move` property.

Number of Kill objects: Counts objects that delete player objects upon contact, effectively acting as lethal hazards.

Number of Sink objects: Measures the number of objects with the `sink` property, which remove themselves and any colliding object. Their presence can be strategically useful or obstructive.

Number of Stopped objects: Counts the number of objects with the `stop` property, which block movement.

2) Distance Min-/Maximizers: These heuristics compute average Manhattan distances between player-controlled objects and other object classes. They are primarily used to encourage movement toward or away from important targets:

Distance to Kill objects: Calculates the average distance between players and killer objects. Often, threats are positioned near valuable goals or key areas, making proximity to them strategically beneficial.

***Distance to Win objects:** Measures how close players are to objects that satisfy winning conditions. If no such objects exist, a default value is returned.

***Distance to Words:** Computes the average distance from players to word tiles. Words are critical for rule manipulation and solving complex puzzles.

***Distance to Push objects:** Evaluates how far players are from pushable objects—typically components of interactable rule statements or obstacles.

3) Meta-Heuristics:

Connectivity: Computes the number of closed rooms in the level, defined as regions isolated from other areas unless a player moves an object. This encourages the agent to open up space and avoid dead ends.

***Stuck IS words:** Returns the number of connecting `IS` words that are stuck—i.e., placed such that they cannot participate in valid rule formation due to board position or obstruction.

***Stuck prefixes:** Measures the number of object-name words (prefixes) that cannot be used in rules, despite their corresponding object existing in the level.

***Stuck suffixes:** Similar to stuck prefixes, but evaluates property or behavior words (e.g., `win`, `stop`) that are no longer positionally valid in rule formation.

***Stuck important suffixes:** A subset of stuck suffixes deemed critical (e.g., `win`, `you`, `defeat`). Blocking access to these typically prevents rule-based progress.

***Newly created rules:** Compares the current rule set with the initial game state and returns the number of novel rules added during play. This allows optimization to either promote experimentation or enforce rule stability.

Path to victory: Checks whether a player object has an unobstructed path to a goal object under current rules. If so, a reward is issued, providing a strong signal for solvable states.

B. Evolutionary Optimization of Search Policies

To guide search in the complex game environment of *Baba Is You*, we optimize search policies using a tree-based representation and evolutionary algorithms. In contrast to prior work by Olson et al. [1], where search policies were expressed as linear combinations of base heuristics, we employ genetic programming to evolve expressive, non-linear tree structures.

Each policy is represented as a syntax tree, where internal nodes denote mathematical operations (+, -, *, sin, ReLU, tanh, -1*) and leaf nodes represent either constant values or base heuristics as described in Section IV-A. When applied to a game state, the tree is evaluated recursively to produce a numerical score. This score determines the priority of the game state during search, affecting the order in which nodes are expanded.

This approach provides greater modeling flexibility than linear scoring functions. It enables the evolution of conditional logic and complex heuristic interactions, potentially allowing agents to better react to complex game states. However, it also introduces a larger search space, making the optimization process more computationally demanding.

To manage this complexity and improve scalability, we evolve a separate search policy for each level. For this process, we use the Python library *pymoo* [8] and its implementation of a genetic algorithm (GA). All internal parameters of the GA are kept at the default. The optimization is run for 50 generations, with 10 individuals per population and a maximal evaluation time of 60 seconds per level. For comparison with Olson et al. [1], we use the same algorithm to optimize linear function search policies. Table I shows the results of the optimization process and indicates a slight advantage of the tree-based approach.

While this per-level specialization accelerates evaluation and improves performance on individual levels, it may reduce the generalizability of the resulting policies. In later stages of our pipeline, we address this limitation by identifying reusable policies and constructing a meta-agent capable of selecting between them (cf. Section V).

Overall, this method provides a trade-off: we achieve strong individual-level performance through per-level optimization while setting the stage for constructing generalized solutions through policy set reduction and classification, as described in subsequent sections.

TABLE I
RESULTS OF THE SEARCH POLICY OPTIMIZATION FOR A SINGLE LEVEL. EACH ENTRY REPORTS THE SMALLEST NUMBER OF SEARCH POLICIES (SET SIZE) REQUIRED TO SOLVE THE GIVEN NUMBER OF LEVELS.

Policy Type	**Nr. of Levels Solved**		
	Training Set	**Test Set**	**Full Set**
Linear	34 / 50	98 / 134	132 / 184
Tree-based	43 / 50	100 / 134	143 / 184

TABLE II
RESULTS OF INTEGER LINEAR PROGRAMMING TO FIND MINIMAL POLICY SETS. EACH ENTRY REPORTS THE SMALLEST NUMBER OF SEARCH POLICIES (SET SIZE) REQUIRED TO SOLVE THE GIVEN NUMBER OF LEVELS.

Policy Type	**Level Set**	**Set Size**	**Levels Solved**
Linear	Training Set	5	129 / 184
Linear	Full Set	12	148 / 184
Tree-based	Training Set	7	130 / 184
Tree-based	Full Set	13	154 / 184

V. CONSTRUCTING A MINIMAL SOLVING SET

Once a search policy has been found for a given level, we test it on all other levels. As a result, we receive an $n \times m$ matrix, where n represents the number of search policies found and m represents the number of levels. If a search policy i solves level j, the matrix is given the entry "1" at the position (i, j), otherwise "0". Ideally, we find at least one search policy per level, such that $n = m$. However, given strict time constraints, this may not be possible.

The matrix is further processed using an integer linear programming (ILP) algorithm to determine the minimum set of search policies required to solve a maximum number of levels. For this purpose, we used the Python library PuLP [9] to construct minimal solving sets for all linear search policies and all tree-based search policies. Specifically, for each variant of search policy (linear and tree-based), and each training configuration (trained only on the training set vs. trained on all levels), we compute a minimal subset of policies such that the union of their solved levels is maximized. The results are summarized in Table II. For both policy types, linear and tree-based policies, we receive set sizes of approximately the same size and expressivity. However, the results for the full set are theoretical in nature. Once the agent faces a new level, we will not have the necessary information on the policy applicability.

VI. ENSEMBLE CONSTRUCTION

While the ILP-derived solving sets represent an upper bound assuming perfect policy selection, practical deployment requires an automated mechanism to select a suitable policy for a previously unseen level. To this end, we train a random forest [10] and a nearest neighbor classifier [11] that predicts the most effective search policy for a given level, using features extracted from the initial game state.

The input features consist of the base heuristics used during policy evolution, evaluated on the initial state of each level. These features capture level-specific properties such as object count, number and type of rules, presence of goals or dangers, and spatial relations between key elements. At inference time, the classifier predicts the most likely policy, which is then used to solve the level. The classifiers' performance is measured

TABLE III
NUMBER OF TEST LEVELS SOLVED USING A CLASSIFIER TRAINED TO SELECT A POLICY FROM THE MINIMAL SOLVING SET (CF. TABLE II) TO SOLVE A GIVEN LEVEL

Policy Type	Classifier	Test Levels Solved
Linear	Nearest Neighbor	97 / 134
Linear	Random Forest	58 / 134
Tree-based	Nearest Neighbor	107 / 134
Tree-based	Random Forest	58 / 134

by the number of levels successfully solved using the returned search policy per level and shown in Table III.

In our comparison, the nearest neighbor classifier clearly outperformed the random forest classifier. This may be due to a high similarity of some levels across the training and test sets, to which the nearest neighbor classifier generalizes better. Furthermore, the tree-based policies showed to generalize better to previously unseen levels. However, neither of these combinations reaches the theoretical upper bound.

As a final test, we compare the resulting nearest neighbor ensembles to the previous approaches. Here, all results have been recorded using a budget of 10000 iterations and 10 seconds per level. The results shown in Table IV highlight a significant performance gain for the trained tree-based encoding and the ensemble of multiple search policies in comparison to previous optimization approaches [1].

VII. CONCLUSION AND FUTURE WORK

In this work, we presented an approach for solving *Baba Is You* levels by evolving search policies using genetic programming. By moving from linear to tree-based policies, we enabled the optimization of more expressive decision functions that better capture the complexity of the game. We further demonstrated how to generalize these policies through the construction of minimal solving sets and the use of classifier-based meta-agents. Our results show that tree-based policies and nearest neighbor selection significantly improve generalization to unseen levels.

Due to the success of the ensemble approach, we aim to further explore how search policy ensembles could be learned. One way to maximize synergies between included search policies could be the use of ensemble reinforcement learning approaches to train neural network policy representations [12] directly. Furthermore, we would like to explore the impact of included search policies in an ensemble to its applicability for generating diverse play-styles in complex strategy games [13], [14]. Additionally, we believe that the use of abstractions has the potential to significantly improve the efficiency of search-based agents in this domain. Various algorithms like Elastic Monte Carlo Tree Search [15] or adaptive abstraction dropping methods [16], as well as hybrid state-action abstraction frameworks [17]–[19], have demonstrated their effectiveness in reducing the complexity of decision-making in large state spaces and may help in tackling the many complex puzzle challenges Baba is You provides.

TABLE IV
COMPARISON OF AGENT PERFORMANCE ACROSS ALL TEST LEVELS WHEN TRAINED USING A TRAINING SET OF 50 LEVELS. VALUES ARE BASED ON 10 RUNS WITH A MAXIMUM BUDGET OF 10000 ITERATIONS AND 10 SECONDS.

Agent	Levels Solved
Tree Ensemble (ours)	**$\approx$ 96 / 134**
Linear Ensemble (ours)	**$\approx$ 84 / 134**
Tree (ours)	**$\approx$ 72 / 134**
Linear [1]	$\approx$ 64 / 134
Default Agent [3]	$\approx$ 65 / 134
BFS [3]	54 / 134
DFS [3]	51 / 134

REFERENCES

[1] C. Olson, L. Wagner, and A. Dockhorn, "Evolutionary optimization of baba is you agents," in *2023 IEEE Congress on Evolutionary Computation (CEC)*, 2023, pp. 1–8.

[2] Y. Shoham and J. Schaeffer, "The FESS algorithm: A feature based approach to single-agent search," in *2020 IEEE Conference on Games (CoG)*, 2020, pp. 96–103.

[3] M. Charity and J. Togelius, "Keke AI competition: Solving puzzle levels in a dynamically changing mechanic space," in *2022 IEEE Conference on Games (CoG)*. IEEE, 2022, pp. 570–575.

[4] M. Charity, "GitHub - MasterMilkX/KekeCompetition: Framework for the Keke Competition - an AI competition for the puzzle game 'Baba is You'," https://github.com/MasterMilkX/KekeCompetition, [Accessed 28-05-2025].

[5] R. Ronge and A. Dockhorn, "Keke is Py Framework," https://github.com/BitDen0m1nat10n/Keke-AI-PY, [Accessed 28-05-2025].

[6] H. Oy, "Baba Is You," https://store.steampowered.com/app/736260/Baba_Is_You/, 2019, [Accessed 30-05-2025].

[7] Hempuli, "Baba Is You (Jam Build)," https://hempuli.itch.io/baba-is-you, 2023, [Accessed 28-05-2025].

[8] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[9] S. Mitchell, "GitHub - coin-or/pulp: A python Linear Programming API," https://github.com/coin-or/pulp, [Accessed 30-05-2025].

[10] A. Parmar, R. Katariya, and V. Patel, "A review on random forest: An ensemble classifier," in *International conference on intelligent data communication technologies and internet of things*. Springer, 2018, pp. 758–763.

[11] E. Fix, *Discriminatory analysis: nonparametric discrimination, consistency properties*. USAF school of Aviation Medicine, 1985, vol. 1.

[12] R. Schmöcker and A. Dockhorn, "Cascade - a sequential ensemble method for continuous control tasks," in *Reinforcement Learning Conference*, 2025.

[13] A. Dockhorn, J. Hurtado-Grueso, D. Jeurissen, L. Xu, and D. Perez-Liebana, "Portfolio search and optimization for general strategy game-playing," in *2021 IEEE Congress on Evolutionary Computation (CEC)*, 2021, pp. 2085–2092.

[14] D. Perez-Liebana, A. Dockhorn, J. H. Grueso, and D. Jeurissen, "The design of "stratega": A general strategy games framework," *arXiv:2009.05643*, pp. 1–7, 2020. [Online]. Available: http://arxiv.org/abs/2009.05643

[15] L. Xu, A. Dockhorn, and D. Perez-Liebana, "Elastic monte carlo tree search," *IEEE Transactions on Games*, 2023.

[16] R. Schmöcker, L. Kampmann, and A. Dockhorn, "Time-critical and confidence-based abstraction dropping methods," in *Proceedings of the 2025 IEEE Conference on Games (CoG)*, 2025, pp. 1–8.

[17] R. Schmöcker and A. Dockhorn, "A survey of non-learning-based abstractions for sequential decision-making," *IEEE Access*, vol. 13, pp. 100 808–100 830, 2025.

[18] A. Dockhorn, J. Hurtado-Grueso, D. Jeurissen, L. Xu, and D. Perez-Liebana, "Game state and action abstracting monte carlo tree search for general strategy game-playing," in *Proceedings of the 2021 IEEE Conference on Games (CoG)*, Aug. 2021, pp. 1–8.

[19] A. Dockhorn and R. Kruse, "State and action abstraction for search and reinforcement learning algorithms," *Artificial Intelligence in Control and Decision-making Systems*, pp. 181–198, 2023.